

littleXML

Návrh a implementácia tabuľkou riadeného LL(1) analyzátora

Dokumentácia riešenia

6. mája 2026

Obsah

1	Zadanie a cieľ riešenia	2
1.1	Pôvodná gramatika	2
2	Transformácia gramatiky	2
2.1	Prepis EBNF formy na pravidlá	2
2.2	Úprava pravidiel	3
2.3	Upravené pravidlá pred kontrolou LL(1)	3
2.4	FIRST/FOLLOW pravých strán pravidiel	4
2.5	Odstránenie konfliktov a výsledná gramatika	6
3	FIRST1 a FOLLOW1 množiny	8
3.1	Použité skratky rozsahov	8
3.2	Vypočítané množiny	8
4	Prechodová tabuľka	8
5	Lexikálny analyzátor	10
5.1	Lexikálne chyby	11
6	Syntaktický analyzátor	11
6.1	Algoritmus	11
7	Zotavenie z chýb	11
7.1	Režim off	11
7.2	Režim panic	12
8	Spustenie programu	12
9	Testovacie vstupy	12
9.1	Pripravené vstupy	12

1 Zadanie a cieľ riešenia

Cieľom projektu je vytvoriť tabuľkou riadený syntaktický analyzátor zhora nadol pre jazyk `littleXML`. Riešenie obsahuje lexikálny analyzátor, LL(1) syntaktický analyzátor, prechodovú tabuľku, výpis krokového logu a jednoduché zotavenie z chýb v režime `panic`.

Program pracuje so súborami. Načíta vstupný súbor, vykoná analýzu a zapíše kompletný report do výstupného súboru. Rovnaký report sa vypíše aj do konzoly.

1.1 Pôvodná gramatika

```
xmldokument := [xmldecl] element.
xmldecl := "<?xml" "version =" vernumb ">".
vernumb := number "." number.
element := emptyelemtag | starttag (words | elements) endtag.
starttag := "<" name ">".
endtag := "</" name ">".
words := {word}.
elements := [element].
emptyelemtag := "<" name ">".
name := (letter | "_" | ":") {namechar}.
namechar := letter | digit | "." | "-" | "_" | ":".
number := digit {digit}.
word := char {char}.
char := letter | digit | alie znaky okrem "<" a ">".
```

Formálne možno gramatiku zapísať ako štvoricu

$$G = (N, T, P, S),$$

kde N je množina neterminálov, T množina terminálov, P množina pravidiel a S štartovací symbol. Štartovací symbol je

$$S = \text{xmldokument}.$$

2 Transformácia gramatiky

Najprv prepíšeme EBNF zápis na obyčajné produkčné pravidlá, potom odstránime spoločné prefixy, vypíšeme medzivýslednú gramatiku a až potom pomocou FIRST/FOLLOW pravých strán pravidiel zistíme konflikty.

2.1 Prepis EBNF formy na pravidlá

Zo zadania najprv odstránime EBNF skratky. Voliteľná časť $[X]$ sa prepíše ako alternatíva s ε , opakovanie $\{X\}$ sa prepíše rekurziou a terminálne rozsahy písmen a číslíc zapíšeme presnými skratkami $A..Z$, $a..z$ a $0..9$.

```
xmldokument -> xmldecl element | element
xmldecl -> <?xml version= vernumb ?>
vernumb -> number . number
element -> emptyelemtag | starttag words endtag | starttag elements endtag
starttag -> < name >
```

```

endtag -> </ name >
words -> word words | epsilon
elements -> element | epsilon
emptyelementtag -> < name />
name -> namestart name_tail
namestart -> letter | _ | :
name_tail -> namechar name_tail | epsilon
namechar -> letter | digit | . | - | _ | :
letter -> A..Z | a..z
number -> digit number_tail
number_tail -> number | epsilon
digit -> 0..9
word -> char word_tail
word_tail -> word | epsilon
char -> letter | digit | . | - | _ | : | ! | @ | #

```

2.2 Úprava pravidiel

Po dosadení `emptyelementtag` a `starttag` do pravidla `element` dostaneme tri alternatívy so spoločným začiatkom:

```

element -> < name /> | < name > words endtag | < name > elements endtag

```

Všetky pravé strany začínajú symbolom `<`. Potrebujeme ľavú faktorizáciu spoločného prefixu `< name`:

```

element -> < name element_tail
element_tail -> /> | > words endtag | > elements endtag

```

V pravidle `element_tail` ešte ostal spoločný prefix `>` pri dvoch alternatívach, odstránime prefix aj tam:

```

element -> < name element_tail
element_tail -> /> | > element_body
element_body -> words endtag | elements endtag

```

Po tejto úprave je rozlíšený prázdny element podľa `/>` a neprázdny element podľa `>`.

2.3 Upravené pravidlá pred kontrolou LL(1)

```

1  xmldokument -> xmldecl element
2  xmldokument -> element
3  xmldecl -> <?xml version= vernumb ?>
4  vernumb -> number . number
5  element -> < name element_tail
6  element_tail -> />
7  element_tail -> > element_body
8  element_body -> words endtag
9  element_body -> elements endtag
10 words -> word words
11 words -> epsilon
12 elements -> element

```

```

13 elements -> epsilon
14 endtag -> </ name >
15 name -> namestart name_tail
16 namestart -> letter
17 namestart -> _
18 namestart -> :
19 name_tail -> namechar name_tail
20 name_tail -> epsilon
21 namechar -> letter
22 namechar -> digit
23 namechar -> .
24 namechar -> -
25 namechar -> _
26 namechar -> :
27 letter -> A..Z | a..z
28 number -> digit number_tail
29 number_tail -> number
30 number_tail -> epsilon
31 digit -> 0..9
32 word -> char word_tail
33 word_tail -> word
34 word_tail -> epsilon
35 char -> letter
36 char -> digit
37 char -> .
38 char -> -
39 char -> _
40 char -> :
41 char -> !
42 char -> @
43 char -> #

```

2.4 FIRST/FOLLOW pravých strán pravidiel

Označme si množinu začiatkov textového slova:

$$W = \{., -, _, :, !, @, \#, 0..9, A..Z, a..z\}.$$

Nasledujúca tabuľka je kompletná tabuľka FIRST/FOLLOW pre pravé strany pravidiel z medzivýslednej gramatiky.

Č.	Pravá strana	FIRST pravej strany	FOLLOW pri ε / poznámka
1	xmldecl element	{<?xml}	—
2	element	{<}	—
3	<?xml version= vernumb ?>	{<?xml}	—
4	number . number	{0..9}	—
5	< name element _tail	{<}	—
6	/>	{/>}	—

Č.	Pravá strana	FIRST pravej strany	FOLLOW pri ε / poznámka
7	> element_body	{>}	–
8	words endtag	$W \cup \{</\}$	konflikt s 9
9	elements endtag	{<, </}	konflikt s 8
10	word words	W	–
11	epsilon	{ ε }	$\text{FOLLOW}_1(\text{words}) = \{</\}$
12	element	{<}	–
13	epsilon	{ ε }	$\text{FOLLOW}_1(\text{elements}) = \{</\}$
14	</ name >	{</}	–
15	namestart name_tail	{_, :, A..Z, a..z}	–
16	letter	{A..Z, a..z}	–
17	–	{_}	–
18	:	{:}	–
19	namechar name_tail	{., -, _, :, 0..9, A..Z, a..z}	–
20	epsilon	{ ε }	$\text{FOLLOW}_1(\text{name_tail}) = \{>, />\}$
21	letter	{A..Z, a..z}	–
22	digit	{0..9}	–
23	.	{.}	–
24	-	{-}	–
25	–	{_}	–
26	:	{:}	–
27	A..Z a..z	{A..Z, a..z}	–
28	digit number_tail	{0..9}	–
29	number	{0..9}	–
30	epsilon	{ ε }	$\text{FOLLOW}_1(\text{number_tail}) = \{?>, .\}$
31	0..9	{0..9}	–
32	char word_tail	W	–
33	word	W	FIRST/FOLLOW konflikt s 34
34	epsilon	{ ε }	$\text{FOLLOW}_1(\text{word_tail}) = W \cup \{</\}$, konflikt s 33
35	letter	{A..Z, a..z}	–
36	digit	{0..9}	–
37	.	{.}	–
38	-	{-}	–
39	–	{_}	–
40	:	{:}	–
41	!	{!}	–
42	@	{@}	–
43	#	{#}	–

Z tabuľky vidíme hlavný $\text{FIRST}_1/\text{FIRST}_1$ konflikt pri netermináli `element_body`. Pravidlá 8 a 9 obsahujú vo FIRST množine rovnaký terminál `</`:

$$\text{FIRST}_1(\text{words endtag}) \cap \text{FIRST}_1(\text{elements endtag}) = \{</\}.$$

To znamená, že do jednej bunky prechodovej tabuľky by sa zapisovali dve pravidlá:

$$M[\text{element_body}, </] = 8$$

a zároveň

$$M[\text{element_body}, </] = 9.$$

Okrem toho je v tabuľke vyznačený aj $\text{FIRST}_1/\text{FOLLOW}_1$ konflikt pri `word_tail`: pravidlo 33 má FIRST množinu W , ale epsilon pravidlo 34 sa zapisuje podľa $\text{FOLLOW}_1(\text{word_tail})$, ktoré tiež obsahuje W :

$$\text{FIRST}_1(\text{word}) \cap \text{FOLLOW}_1(\text{word_tail}) = W.$$

2.5 Odstránenie konfliktov a výsledná gramatika

Pred úpravou sme mali:

```
element_tail -> /> | > element_body
element_body -> words endtag | elements endtag
words -> word words | epsilon
elements -> element | epsilon
```

Konflikt vznikol preto, že prázdny obsah sa dal odvodiť dvoma rôznymi cestami:

```
element_body -> words endtag
words -> epsilon
```

a zároveň:

```
element_body -> elements endtag
elements -> epsilon
```

Preto pri vstupe `</` nebolo jasné, či má parser použiť vetvu cez `words`, alebo vetvu cez `elements`. Prázdny obsah však v jazyku potrebujeme iba raz. Preto obsah elementu rozdelíme priamo na tri prípady:

```
element_tail -> /> | > element_content endtag
element_content -> word | element | epsilon
```

- ak obsah začína znakom z množiny W , použije sa pravidlo `element_content -> word`,
- ak obsah začína terminálom `<`, použije sa pravidlo `element_content -> element`,
- ak obsah začína terminálom `</`, použije sa pravidlo `element_content -> epsilon`.

Pre neterminál `element_content` potom dostaneme:

$$\text{FIRST}_1(\text{word}) = W, \quad \text{FIRST}_1(\text{element}) = \{<\}, \quad \text{FOLLOW}_1(\text{element_content}) = \{</\}.$$

FIRST/FIRST konflikt zmizne, pretože:

$$W \cap \{<\} = \emptyset.$$

FIRST/FOLLOW konflikt pri epsilon pravidle tiež nevznikne, pretože epsilon pravidlo sa zapisuje podľa $\text{FOLLOW}_1(\text{element_content})$, teda iba pod $</$:

$$(\text{FIRST}_1(\text{word}) \cup \text{FIRST}_1(\text{element})) \cap \text{FOLLOW}_1(\text{element_content}) = (W \cup \{<\}) \cap \{</\} = \emptyset.$$

Tým sa odstráni aj pôvodný konflikt pri word_tail . V medzivýslednej gramatike sa po jednom word mohlo podľa pravidla $\text{words} \rightarrow \text{word words}$ začať ďalšie word , preto $\text{FOLLOW}_1(\text{word_tail})$ obsahovalo množinu W . Po odstránení words sa word používa priamo v pravidle $\text{element_content} \rightarrow \text{word}$; za ním nasleduje už iba endtag . Preto platí:

$$\text{FOLLOW}_1(\text{word_tail}) = \{</\}$$

a teda:

$$\text{FIRST}_1(\text{word}) \cap \text{FOLLOW}_1(\text{word_tail}) = W \cap \{</\} = \emptyset.$$

Výsledná úprava je zlúčenie prázdneho obsahu do jednej vetvy a odstránenie nejednoznačného delenia textu na viacero word pravidiel.

Výsledná LL(1) gramatika použitá v implementácii je:

```

xmldokument -> xmldecl element | element
xmldecl -> <?xml version= vernumb ?>
vernumb -> number . number
element -> < name element_tail
element_tail -> /> | > element_content endtag
element_content -> word | element | epsilon
endtag -> </ name >
name -> namestart name_tail
namestart -> letter | _ | :
name_tail -> namechar name_tail | epsilon
namechar -> letter | digit | . | - | _ | :
letter -> A..Z | a..z
number -> digit number_tail
number_tail -> number | epsilon
digit -> 0..9
word -> char word_tail
word_tail -> word | epsilon
char -> letter | digit | . | - | _ | : | ! | @ | #

```

Použité rozsahy sú skratky pre konkrétne terminály:

$$A..Z = \{A, \dots, Z\}, \quad a..z = \{a, \dots, z\}, \quad 0..9 = \{0, \dots, 9\}.$$

Riešenie nepoužíva skratkové tokeny **NAME** a **TEXT**. Meno tagu, číslo verzie a textové slovo sa skladajú až syntaktickou analýzou z detailných terminálov. V implementačnej tabuľke sa rozsahy iba technicky mapujú na peek triedy **LETTER** a **NUMBER**.

3 FIRST1 a FOLLOW1 množiny

3.1 Použité skratky rozsahov

$$\begin{aligned} 0..9 &= \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, \\ A..Z &= \{A, B, C, \dots, X, Y, Z\}, \\ a..z &= \{a, b, c, \dots, x, y, z\}. \end{aligned}$$

3.2 Vypočítané množiny

Neterminál	FIRST ₁	FOLLOW ₁
xmldokument	{<?xml, <}	{\$}
xmldecl	{<?xml}	{<}
vernumb	{0,...,9}	{?>}
element	{<}	{\$, </}
element_tail	{/>, >}	{\$, </}
element_content	{ ε , <, ., -, _, :, !, @, #, 0..9, A..Z, a..z}	{</}
endtag	{</}	{\$, </}
name	{_, :, A..Z, a..z}	{>, />}
namestart	{_, :, A..Z, a..z}	{>, />, ., -, _, :, 0..9, A..Z, a..z}
name_tail	{ ε , ., -, _, :, 0..9, A..Z, a..z}	{>, />}
namechar	{., -, _, :, 0..9, A..Z, a..z}	{>, />, ., -, _, :, 0..9, A..Z, a..z}
letter	{A..Z, a..z}	{>, </, />, ., -, _, :, !, @, #, 0..9, A..Z, a..z}
number	{0,...,9}	{?>, .}
number_tail	{ ε , 0,...,9}	{?>, .}
digit	{0,...,9}	{?>, >, </, />, ., -, _, :, !, @, #, 0..9, A..Z, a..z}
word	{., -, _, :, !, @, #, 0..9, A..Z, a..z}	{</}
word_tail	{ ε , ., -, _, :, !, @, #, 0..9, A..Z, a..z}	{</}
char	{., -, _, :, !, @, #, 0..9, A..Z, a..z}	{</, ., -, _, :, !, @, #, 0..9, A..Z, a..z}

4 Prechodová tabuľka

Prechodová tabuľka je uložená v súbore `table.csv`. Riadky sú neterminály, stĺpce sú vstupné terminály a bunky obsahujú čísla pravidiel zo súboru `rules.py`. Položka tabuľky má tvar:

$$M[A, a] = i,$$

kde A je neterminál na vrchu zásobníka, a je aktuálny vstupný symbol a i je číslo pravidla. Parser následne zo slovníka pravidiel vyberie produkciu

$$\text{RULES}[i] = A \rightarrow \alpha.$$

Neterminál	Peek	Pravidlo z rules.py
xmldokument	<?xml	xmldecl element
xmldokument	<	element
xmldecl	<?xml	<?xml version= vernumb ?>
vernumb	NUMBER	number . number
element	<	< name element_tail
element_tail	/>	/>
element_tail	>	> element_content endtag
element_content	LETTER	word
element_content	NUMBER	word
element_content	.	word
element_content	-	word
element_content	_	word
element_content	:	word
element_content	!	word
element_content	@	word
element_content	#	word
element_content	<	element
element_content	</	epsilon
endtag	</	</ name >
name	LETTER	namestart name_tail
name	-	namestart name_tail
name	:	namestart name_tail
namestart	LETTER	letter
namestart	-	-
namestart	:	:
name_tail	LETTER	namechar name_tail
name_tail	NUMBER	namechar name_tail
name_tail	.	namechar name_tail
name_tail	-	namechar name_tail
name_tail	_	namechar name_tail
name_tail	:	namechar name_tail
name_tail	>	epsilon
name_tail	/>	epsilon
namechar	LETTER	letter
namechar	NUMBER	digit
namechar	.	.
namechar	-	-
namechar	_	-
namechar	:	:
letter	LETTER	LETTER
number	NUMBER	digit number_tail

Neterminál	Peek	Pravidlo z rules.py
number_tail	NUMBER	number
number_tail	?>	epsilon
number_tail	.	epsilon
digit	NUMBER	NUMBER
word	LETTER	char word_tail
word	NUMBER	char word_tail
word	.	char word_tail
word	-	char word_tail
word	_	char word_tail
word	:	char word_tail
word	!	char word_tail
word	@	char word_tail
word	#	char word_tail
word_tail	LETTER	word
word_tail	NUMBER	word
word_tail	.	word
word_tail	-	word
word_tail	_	word
word_tail	:	word
word_tail	!	word
word_tail	@	word
word_tail	#	word
word_tail	</	epsilon
char	LETTER	letter
char	NUMBER	digit
char	.	.
char	-	-
char	_	_
char	:	:
char	!	!
char	@	@
char	#	#

5 Lexikálny analyzátor

Lexikálny analyzátor je implementovaný v triede `Tokenizer` pomocou regulárnych výrazov. Prechádza vstup zľava doprava, v každom kroku vyberie najdlhší regulárny výraz, ktorý sa zhoduje na aktuálnej pozícii, a produkuje tokeny.

Regexový tokenizer rozpoznáva pevné viacznakové a jednoznakové terminály:

```
<?xml version= ?> </ /> < > . - : _ ! @ #
```

Ak sa nenašiel pevný terminál, jeden vstupný znak sa klasifikuje ako:

LETTER, ak $c \in \{A..Z, a..z\}$,

alebo

NUMBER, ak $c \in \{0..9\}$.

5.1 Lexikálne chyby

Ak sa nájde znak, ktorý nepatrí medzi povolené terminály ani do tried `LETTER/NUMBER`, nastane jeden z dvoch scenárov zotavenia. V režime `off` analýza končí pri prvej lexikálnej chybe. V režime `panic` lexikálny analyzátor preskočí neznámy úsek po najbližší rozpoznateľný token alebo biely znak.

6 Syntaktický analyzátor

Syntaktický analyzátor je implementovaný v triede `Parser`. Ide o tabuľkou riadený LL(1) analyzátor so zásobníkom.

6.1 Algoritmus

Na začiatku je na zásobníku:

$$[\$, S] = [\$, \text{xmldokument}].$$

V každej iterácii parser vykoná tieto kroky:

1. Zoberie symbol z vrchu zásobníka.
2. Pozrie aktuálny vstupný token, v reporte označený ako `Peek`.
3. Ak je vrch zásobníka terminál a zhoduje sa s `Peek`, token sa spotrebuje.
4. Ak je vrch zásobníka neterminál, použije sa položka prechodovej tabuľky $M[A, a]$, ktorá obsahuje číslo pravidla.
5. Pravá strana vybraného očíslovaného pravidla sa vloží na zásobník v opačnom poradí.
6. Ak zásobník aj vstup skončia na `$` bez diagnostiky, vstup je syntakticky správny.

Jedna iterácia v reporte obsahuje:

Zasobnik pred
Vstup pred
Peek
Krok
Zasobnik po
Vstup po

7 Zotavenie z chýb

Program podporuje dva režimy zotavenia:

7.1 Režim off

V režime `off` analýza končí pri prvej lexikálnej alebo syntaktickej chybe.

7.2 Režim panic

V režime `panic` sa parser pokúsi pokračovať po syntaktickej chybe. Ak je na vrchu zásobníka neterminál A a pre dvojicu (A, a) neexistuje pravidlo v tabuľke, použije sa synchronizačná množina:

$$\text{sync}(A) = \text{FOLLOW}_1(A).$$

Parser preskakuje vstupné tokeny, kým nenájde token zo $\text{sync}(A)$ alebo koniec vstupu $\$$. Následne zahodí problémový neterminál zo zásobníka a pokračuje v analýze.

8 Spustenie programu

Program sa dá spustiť dvoma spôsobmi. Bežný spôsob je bez argumentov:

```
python .\projekt\main.py
```

Výstupný report obsahuje:

- vstupný súbor a režim zotavenia,
- pôvodný vstup,
- pravidlá z `rules.py`,
- zoznam tokenov,
- kompletný log syntaktického analyzátoru,
- postupnosť použitých očíslovaných pravidiel,
- problémy validácie,
- výsledok SYNTAX OK alebo SYNTAX ERROR.

9 Testovacie vstupy

9.1 Pripravené vstupy

Súbor	Výsledok	Účel
01_valid_empty_element.xml	OK	Testuje, že analyzátor prijme prázdny element zapísaný pomocou <code></></code> .
02_valid_xml_declaration.xml	OK	Testuje, že pred koreňovým elementom môže byť platná XML deklarácia s verziou.
03_valid_text_content.xml	OK	Testuje, že element môže obsahovať jedno textové slovo zo znakov povolených pravidlom <code>char</code> .
04_valid_one_nested_element.xml	OK	Testuje, že element môže obsahovať práve jeden vnorený element.

Súbor	Výsledok	Účel
05_valid_nested_text.xml	OK	Testuje, že gramatika povoľuje opakované vnorenie elementov, ak je v každom obsahu najviac jeden vnorený element.
07_invalid_two_nested_children.xml	ERROR	Testuje, že gramatika odmietne dva súrodenecké vnorené elementy v jednom obsahu.
08_invalid_text_then_element.xml	ERROR	Testuje, že jeden obsah elementu nemôže kombinovať textové slovo a vnorený element.
09_invalid_bad_version.xml	ERROR	Testuje, že verzia v XML deklarácii musí mať tvar number.number .
10_invalid_name_starts_with_digit.xml	ERROR	Testuje, že názov elementu nesmie začínať číslicou.
11_invalid_lexical_recovery.xml	ERROR	Testuje, že lexikálny analyzátor zachytí nepovolený znak a v režime panic sa vie posunúť ďalej.